

Декодирование потока — инженерный разбор

Тот же материал, что в академической монографии `DecoderGuide.Academic.ru.md`, но без формализмов ради формализмов: короче записи, больше «почему сделано именно так» и «как это ведёт себя на реальном потоке». Нумерация разделов в обеих версиях совпадает, так что ссылки взаимозаменяемы. Вводный разбор «с нуля» — `DecoderGuide.Tutorial.ru.md`; контекст — в `AlgorithmGuide.ru.md` (разделы 5–9); прямая половина кодека — `EncoderGuide.Engineer.ru.md`.

1. Что мы вообще разбираем

`FileDecoder` принимает поток (Base64-текст, сырые байты, `Stream`, файл) и превращает его в слоты приёма — по одному на опознанный файл. Сборка из слота — отдельный цех (`AssemblyGuide.*`); здесь только конвейер доставки: источник → фильтр Base64 (txt) → сканер скользящего окна → накопитель. Два сканера (txt/bin) живут одновременно и удерживают пропущенные данные между вызовами `Scan`.

2. Архитектура конвейера

Всё построено на паре интерфейсов `IDataSource` / `IDataProcessor` (сборка `DataShield.Interfaces`): источник читает блоки и объявляет `DataReady` с делегатом вычитки; обработчик `Attach` — к источнику, делает своё дело и сам является источником для следующего. `DataProcessorBase` даёт: выходной буфер с порогом выдачи, два режима вывода (байтовый `Emit` и пакетный `EmitPacket` — пакеты не режутся границами выдачи), каскадные `Start/Stop/Complete`, всё под `SynRoot`. Канал `Error`: исключение источника всплывает вверх делегированием, `Completion` фолтится той же ошибкой — сбои ввода-вывода не глощаются.

3. Входные форматы

`Scan(IEnumerable<string>)` — строки конкатенируются, UTF8, дальше txt-конвейер; короче 100 символов — выходим молча. `Scan(byte[])` — бинарный вход; короче 75 — молча. `Scan(Stream, OutputFormat)` — явный формат, длина для прогресса берётся у позиционируемых. `PacketIO.ScanFile` смотрит расширение: `.txt` → Base64, `.bin` → Binary, нераспознанное → Base64.

4. Фильтр Base64

`ByteRangeFilter.CreateBase64()` — таблица `bool[256]`, пропускаются `A-Z`, `a-z`, `0-9`, `+`, `/`. `=` не нужен: строка пакета — ровно 100 символов без добавки. Переводы строк, декоративные обрамления и мусор вне алфавита выбрасываются до сканера. В binary-режиме фильтра в цепочке нет.

5. Скользящее окно: прямой проход

`SlidingWindowScanner(windowSize, handler)`: окно 100 (txt) или 75 (bin); `handler` возвращает продвижение и опционально пакет. Неудача → сдвиг на 1 байт (начало пакета не проскочит); успех → выдача пакета и прыжок на размер окна (быстрая дорога). Сканер **удерживает весь поток** (`_retained`) — это плата за перепривязку (раздел 10). Прогресс наружу — событие `ConsumedAdvanced` по позиции прямого прохода.

6. Распознавание пакетов

`FileDecoder.TxtWindow`: 100 символов → `Convert.TryFromBase64Chars` → 75 байт; сбой декода или нераспознавание → сдвиг 1. `BinWindow`: сырые 75 байт сразу на проверку. Решает `StreamProcessor.Recognizes`: заголовок — автономная проверка `VerifyHeaderPacket` (H5 по первым 51 байтам); сектор — перебор слотов: номер в `0..T-1` и `VerifySectorPacket(packet, slot.HeaderHash)`. Ложные срабатывания: 2^{-192} на заголовок, 2^{-72} на сектор.

7. Накопитель: куски → пакеты → слоты

`StreamProcessor.ProcessChunk`: 75-байтный `_pending` буфер собирает пакеты из кусков произвольной границы. Готовый пакет: заголовок → `AcceptHeader` (побайтовое сравнение со слотами: совпал → `IncrementHeaderCount`, новенький → `HeaderContent.ReadFrom` + H5 + новый `ReceptionSlot`, событие `HeaderAccepted` после снятия блокировки); сектор → во все подходящие слоты (`SectorMatches` → `AddSector(num, payload-копия)`). Метрики наружу: `FileCount`, `TotalReceivedSectorCount`, `TotalReceivedSectorCopyCount`, `TotalCollisionSectorCount`.

8. Слоты и версии

`ReceptionSlot` — словарь «номер → список версий по убыванию счётчика». Вся логика версий, коллизий и сборки — `AssemblyGuide.Engineer.ru.md`. Для конвейера важно одно: сектор, подходящий нескольким слотам, попадает в каждый (у файлов разные H5, совпадение практически исключено).

9. Многофайловость

Слотов сколько, сколько разных заголовков в потоке. Каждый сектор проверяется против всех слотов; подтверждения отдельные. `Slots` — снимок под блокировкой, читать можно параллельно с приёмом.

10. Адресная перепривязка

Проблема: сектор до первого заголовка неопознаваем (нет сида). Решение: при событии `HeaderAccepted` декодер просит оба сканера перепривязать удержанные данные — повторный проход с `RebindWindow`: декод (txt), номер в диапазоне нового `T`, `VerifySectorPacket` с новым `H5`. Активный сканер выполняет запрос отложено (граница — `_flushedPos`); простаивающий — немедленно по всему пройденному объёму, с временным подключением накопителя к нему. Перепривязка адресная: чужие слоты не затрагиваются.

11. Повторное сканирование

Две жёсткие правила повторного прохода. Первое — исчерпываемость: окно проверяется на **каждой** позиции региона, без прыжков; прямой проход после успеха прыгает на окно и на повреждениях может перескочить перекрывающийся пакет — повторный проход закрывает эти дыры. Второе — граница: перепривязка идёт только до позиции выдачи прямого прохода (`_flushedPos`, фиксируется в `OnDelivering` до вызова обработчиков), иначе задвоение подтверждений. Запросы, пришедшие во время прохода, копят в очереди и исполняются после.

12. Финализация конвейера

`RunPipeline` закрывает цепочку по порядку: `filter?.Complete()` → `scanner.Complete()` → `processor.Complete()`. Финальная выдача сканера поднимает границу и запускает отложенные перепривязки — покрывает пропуски прямого прохода. Пропуск `Complete` = потерянные хвосты и недостающие перепривязки; полнота приёма гарантирована только после него.

13. Прогресс и отмена

Фаза: `HeaderSearch`, пока `FileCount == 0`, дальше `SectorSearch`; процент — потреблено/итого по `ConsumedAdvanced`; терминал — `Done` (100). Отмена: проверка в колбэке прогресса и в ожидании `Completion`; исключение наружу, накопленное состояние сохраняется.

14. Сборка

`TryAssemble(HeaderContent)`: сериализация заголовка → побайтовый поиск слота → `slot.TryAssemble(_rs, progress, ct)`. RS-адаптер создаётся декодером один на всё время жизни. Дальше — `AssemblyGuide.Engineer.ru.md`.

15. IO-модуль и ошибки

Источники: `ByteArraySource`, `StreamSource`, `FileSource`. Приёмники результата: `StreamDataWriter`, `FileDataWriter`, `ByteListWriter`, `PreallocatedBufferWriter`. Канал ошибок: `BufferedSourceBase` ловит исключение чтения, фиксирует в `Error`, останавливается, `Completion` фолтится; обработчики делегируют `Error` вверх. Итог: сбой диска доезжает до вызывающего кода честным исключением из `Completion.Wait`, а не пустым результатом.

16. Гарантии и пределы

Гарантировано:

1. Оpozнание только по точному хешу: мусор/подделка в слоты не попадает.
2. Шаг 1 при неудаче + исчерпывающий повторный проход: корректные пакеты не теряются из-за мусора и рассинхронизации.
3. Сектор-раньше-заголовка подбирается перепривязкой; подтверждения не задваиваются (граница выдачи).
4. Ошибки ввода-вывода не теряются (`Error` / `Completion`).
5. Состояние слотов читается потокобезопасно параллельно с приёмом.

Пределы: память сканера растёт линейно по входу (весь поток удерживается — плата за перепривязку); стоимость порчи одного символа Base64 — до ~99 сдвигов окна; перепривязка покрывает только удержанные данные текущего вызова жизни декодера.